# Homework #6

Due: 11:59PM EST, May 1 2026

## Sequential Models and Decision Making

### Introduction

In this assignment, you will:

- Work with hidden Markov models in the continuous-state setting of Kalman filters.
- Implement policy iteration and value iteration for a Gridworld Markov decision processes.
- Implement a Q-learning agent to play Swingy Monkey.
- Analyze autoregressive models: decoding, KV caching, and speculative decoding.
- Think about how reward function choices in deployed reinforcement learning systems might contribute to social harms like political polarization (embedded ethics question).

### Resources and Submission Instructions

Please type your solutions after the corresponding problems using this LaTeX template, and start each problem on a new page.

Please submit the writeup PDF to the Gradescope assignment 'HW6'. Remember to assign pages for each question. **You must include any plots in your writeup PDF.** Please submit your LaTeXfile and code files to the Gradescope assignment 'HW6 - Supplemental.' The supplemental files will only be checked in special cases, e.g. honor code issues, etc. Your files should be named in the same way as we provide them in the repository, e.g. `hw6.pdf`, etc.

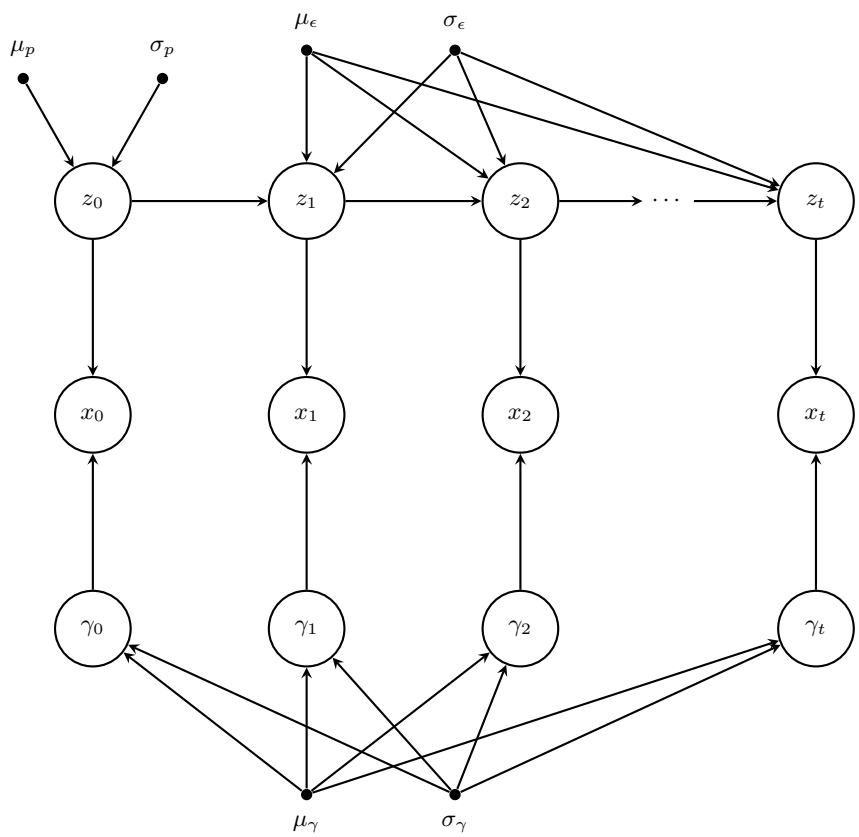
**Problem 1** (Hidden Markov Models, 15 pts)

In this problem, you will be working with one-dimensional Kalman filters, which are *continuous-state* Hidden Markov Models. Let $z_0, z_1, \cdots, z_t$ be the hidden states of the system and $x_0, x_1, \cdots, x_t$ be the observations produced. Then, state transitions and emissions of observations work as follows:

$$\begin{aligned} z_{t+1} &= z_t + \epsilon_t \\ x_t &= z_t + \gamma_t \end{aligned}$$

where $\epsilon_t \sim N(0, \sigma_\epsilon^2)$ and $\gamma_t \sim N(0, \sigma_\gamma^2)$. The value of the first hidden state follows the distribution $z_0 \sim N(\mu_p, \sigma_p^2)$.

The below graphical model may help you understand the structure of the probabilistic model for our one-dimensional Kalman filter.




1. In this first part, we will walk through the derivation of the conditional distribution of $z_t|(x_0, \cdots, x_t)$.

   (a) How does the quantity $p(z_t|x_0, \cdots, x_t)$ relate to $\alpha_t(z_t)$ and $\beta_t(z_t)$ from the forward-backward algorithm for HMMs? What is the operation we are performing called?

   (b) The above quantity $p(z_t|x_0, \cdots, x_t)$ is the PDF for a Normal distribution with mean $\mu_t$ and variance $\sigma_t^2$. We start our derivation of $\mu_t$ and $\sigma_t^2$ by writing:

$$p(z_t|x_0, \cdots, x_t) \propto p(x_t|z_t)p(z_t|x_0, \cdots x_{t-1})$$

What is $p(x_t|z_t)$ equal to?

(c) Suppose we are given the mean and variance of the distribution $z_{t-1}|(x_0, \cdots, x_{t-1})$ as $\mu_{t-1}$, $\sigma^2_{t-1}$. What is $p(z_t|x_0, \cdots x_{t-1})$ equal to?

**Hint 1**: Start by marginalizing out over $z_{t-1}$.

**Hint 2**: You may cite the fact that

$$\int N(y-x; \mu_a, \sigma_a^2) N(x; \mu_b, \sigma_b^2) dx = N(y; (\mu_a + \mu_b), (\sigma_a^2 + \sigma_b^2))$$

(d) Combine your answers from parts (b) and (c) to get a final expression for $p(z_t|x_0, \cdots, x_t)$. Report the mean $\mu_t$ and variance $\sigma_t^2$ of this Normal.

**Hint 1**: Rewrite $N(x_t; z_t, \sigma_\gamma^2)$ as $N(z_t; x_t, \sigma_\gamma^2)$.

**Hint 2**: You may cite the fact that

$$N(x; \mu_a, \sigma_a^2) N(x; \mu_b, \sigma_b^2) \propto N\left(x; \frac{\sigma_b^2}{\sigma_a^2 + \sigma_b^2}\mu_a + \frac{\sigma_a^2}{\sigma_a^2 + \sigma_b^2}\mu_b, \left(\frac{1}{\sigma_a^2} + \frac{1}{\sigma_b^2}\right)^{-1}\right)$$

2. In a sentence or two, interpret $\mu_t$ in terms of how it combines observations from the past with the current observation.

## Solution

Your solution here.

**Problem 2** (Policy and Value Iteration, 15 pts)

You have a robot that you wish to collect two parts in an environment and bring them to a goal location. There are also parts of the environment that you wish the robot avoid to reduce wear on the floor.

**Environment model.** Eventually, you settle on the following way to model the environment as a Gridworld. The "states" in Gridworld are represented by locations in a two-dimensional space. Here we show each state and its reward:

| | | | | |
|---|---|---|---|---|
| R=4 | R=0 | R=-10 | R=0 | R=20 |
| R=0 | R=0 | R=-50 | R=0 | R=0 |
| START R=0 | R=0 | R=-50 | R=0 | R=50 |
| R=0 | R=0 | R=-20 | R=0 | R=0 |

The set of actions is {N, S, E, W}, which corresponds to moving north (up), south (down), east (right), and west (left) on the grid. Taking an action in Gridworld does not always succeed with probability 1; instead the agent has probability 0.1 of "slipping" into a state on either side, but not backwards. For example, if the agent tries to move right from START, it succeeds with probability 0.8, but the agent may end up moving up or down with probability 0.1 each. Also, the agent cannot move off the edge of the grid, so moving left from START will keep the agent in the same state with probability 0.8, but also may slip up or down with probability 0.1 each. Lastly, the agent has no chance of slipping off the grid, so moving up from START results in a 0.9 chance of success with a 0.1 chance of moving right.

Also, the agent does not receive the reward of a state immediately upon entry, but instead only after it takes an action at that state. For example, if the agent moves right four times (deterministically, with no chance of slipping) the rewards would be +0, +0, -50, +0, and the agent would reside in the +50 state. Regardless of what action the agent takes here, the next reward would be +50.

**Action items.** In this problem, you will first implement policy and value iteration in this setting and discuss the policies that you find. Next, you will interrogate whether this approach to modeling the original problem was appropriate.

Your job is to implement the following three methods in file `homework6.ipynb`. Please use the provided helper functions `get_reward` and `get_transition_prob` to implement your solution. *Do not use any outside code.*

**Important note.** The state space is represented using integers, which range from 0 (the top left) to 19 (the bottom right). Therefore both the policy `pi` and the value function `V` are 1-dimensional arrays of length `num_states = 20`. Your policy and value iteration methods should only implement one update step of the iteration - they will be repeatedly called by the provided `learn_strategy` method to learn and display the optimal policy. You can change the number of iterations that your code is run and

displayed by changing the `max_iter` and `print_every` parameters of the `learn_strategy` function calls at the end of the code.

Note that we are doing infinite-horizon planning to maximize the expected reward of the traveling agent. For parts 1-3, set discount factor $\gamma = 0.7$.

1. First, on policy iteration. Note that you do not need submit anything in your write-up for 1(a) and 1(b).

   (a) Implement function `policy_evaluation`. Your solution should learn value function $V$, either using a closed-form expression or iteratively using convergence tolerance `theta = 0.0001` (i.e., if $V^{(t)}$ represents $V$ on the $t$-th iteration of your policy evaluation procedure, then if $|V^{(t+1)}[s] - V^{(t)}[s]| \leq \theta$ for all $s$, then terminate and return $V^{(t+1)}$.)

   (b) Implement function `update_policy_iteration` to update the policy `pi` given a value function `V` using **one step** of policy iteration.

   (c) Set `max_iter = 4`, `print_every = 1` to show the learned value function and the associated policy for the first 4 policy iterations. Do not modify the plotting code. Please fit all 4 plots onto one page of your writeup.

   (d) Set `ct = 0.01` and increase `max_iter` such that the algorithm converges. Include a plot of the final learned value function and policy. How many iterations does it take to converge? Now if we try `ct = 0.001` and `ct = 0.0001`, how does this affect the number of iterations until convergence?

2. Next, on value iteration.

   (a) Implement function `update_value_iteration`, which performs **one step** of value iteration to update `V`, `pi`.

   (b) Set `max_iter = 4`, `print_every = 1` to show the learned value function and the associated policy for the first 4 value iterations. Do not modify the plotting code. Please fit all 4 plots onto one page of your writeup.

   (c) Set `ct = 0.01` and increase `max_iter` such that the algorithm converges. Include a plot of the final learned value function and policy. How many iterations does it take to converge? Now try `ct = 0.001` and `ct = 0.0001`. How does this affect the number of iterations until convergence?

3. Plot the learned policy with each of $\gamma \in (0.6, 0.7, 0.8, 0.9)$. Include all 4 plots in your writeup. Describe in one paragraph how the observed policies differ and possible explanations.

4. Now suppose that the game ends at any state with a positive reward, i.e. it immediately transitions you to a new state with zero reward that you cannot transition away from. What do you expect the optimal policy to look like, as a function of gamma? Numerical answers are not required, intuition is sufficient.

5. In this problem, we came up with a model for the problem, solved it, and then we had a policy to use on the real robot. What is the value of the approach we took, compared to, e.g., using RL on the robot to identify a policy that achieved your objective? What are some limitations of our modeling approach (which may also apply to RL)?

## Solution

Your solution here.

**Problem 3** (Reinforcement Learning, 20 pts)

In 2013, the mobile game *Flappy Bird* took the world by storm. You'll be developing a Q-learning agent to play a similar game, *Swingy Monkey* (See Figure **??**). In this game, you control a monkey that is trying to swing on vines and avoid tree trunks. You can either make him jump to a new vine, or have him swing down on the vine he's currently holding. You get points for successfully passing tree trunks without hitting them, falling off the bottom of the screen, or jumping off the top. There are some sources of randomness: the monkey's jumps are sometimes higher than others, the gaps in the trees vary vertically, the gravity varies from game to game, and the distances between the trees are different. You can play the game directly by pushing a key on the keyboard to make the monkey jump. However, your objective is to build an agent that *learns* to play on its own.

**Task:** Your task is to use Q-learning to find a policy for the monkey that can navigate the trees. The `homework6_soln.ipynb` file contains starter code for setting up your learner that interacts with the game. This is the **only code file** you need to modify. At the beginning of the code, you will import the `SwingyMonkey` class, which is the implementation of the game that has already been completed for you. Note that by default we have you import this class from the file `SwingyMonkeyNoAnimation.py`, which allows you to speed up testing. To actually see the game animation, you can instead import from `SwingyMonkey.py`. Additionally, we provide a video of the staff Q-Learner playing the game at https://youtu.be/xRD6xBQbauw. It figures out a reasonable policy in a few iterations.

You'll be responsible for implementing the Python function `action_callback`. The action callback will take in a dictionary that describes the current state of the game and return an action for the next time step. This will be a binary action, where 0 means to swing downward and 1 means to jump up. The dictionary you get for the state looks like this:

```
{ 'score': <current score>,
  'tree': { 'dist': <pixels to next tree trunk>,
            'top': <height of top of tree trunk gap>,
            'bot': <height of bottom of tree trunk gap> },
  'monkey': { 'vel': <current monkey y-axis speed>,
              'top': <height of top of monkey>,
              'bot': <height of bottom of monkey> }}
```

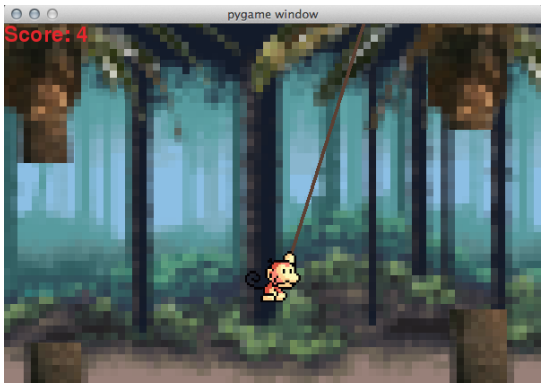
All of the units here (except score) will be in screen pixels. Figure **??** shows these graphically. Note that since the state space is very large (effectively continuous), the monkey's relative position needs to be discretized into bins. The pre-defined function `discretize_state` does this for you.

**Requirements**

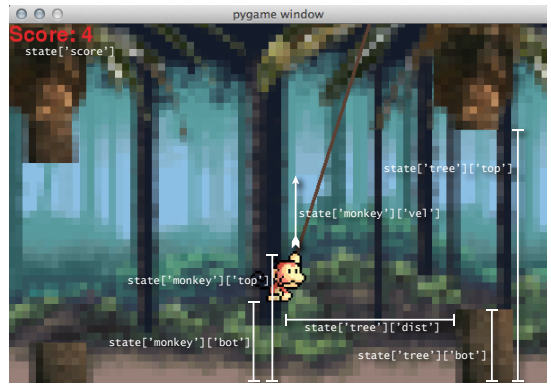
*Code*: First, you should implement Q-learning with an $\epsilon$-greedy policy yourself. You can increase the performance by trying out different parameters for the learning rate $\alpha$, discount rate $\gamma$, and exploration rate $\epsilon$. *Do not use outside RL code for this assignment.* Second, you should use a method of your choice to further improve the performance. This could be inferring gravity at each epoch (the gravity varies from game to game), updating the reward function, trying decaying epsilon greedy functions (this method in particular is highly recommended), changing the features in the state space, and more. The minimum expectation to have an agent that reaches scores over 50 at least once before the 100th epoch.

*Evaluation*: In 1-2 paragraphs, explain how your agent performed and what decisions you made and why. Make sure to provide evidence where necessary to explain your decisions. You must include in your write up at least one plot or table that details the performances of parameters tried (i.e. plots of score vs. epoch number for different parameters).

*Note*: Note that you can simply discretize the state and action spaces and run the Q-learning algorithm. There is no need to use complex models such as neural networks to solve this problem.




(a) SwingyMonkey Screenshot




(b) SwingyMonkey State

Figure 1: (a) Screenshot of the Swingy Monkey game. (b) Interpretations of various pieces of the state dictionary.

# Solution

Your solution here.

**Problem 4** (Autoregressive Models, 20 pts)

An autoregressive model factorizes a joint distribution over a sequence $\mathbf{x} = (x_1, \ldots, x_T)$ using the chain rule of probability:

$$p(x_1, \ldots, x_T) \;=\; \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

In this problem you will examine three consequences of the sequential structure of autoregressive generation. Part 1 concerns the algorithmic cost of finding the most probable sequence. Part 2 concerns the computational cost of each generation step and how the causal structure of a Transformer permits caching. Part 3 introduces speculative decoding, a technique used in modern LLM inference to partially parallelize generation.

**Part 1: Decoding autoregressive models.**

Consider the following autoregressive model over sequences $\mathbf{x} = (x_1, x_2, x_3)$ with each $x_t \in \{A, B\}$:

$$\begin{aligned}
p(x_1 = A) &= 0.6, & p(x_1 = B) &= 0.4 \\
p(x_2 = A \mid x_1 = A) &= 0.7, & p(x_2 = B \mid x_1 = A) &= 0.3 \\
p(x_2 = A \mid x_1 = B) &= 0.1, & p(x_2 = B \mid x_1 = B) &= 0.9 \\
p(x_3 = A \mid A, A) &= 0.6, & p(x_3 = B \mid A, A) &= 0.4 \\
p(x_3 = A \mid A, B) &= 0.5, & p(x_3 = B \mid A, B) &= 0.5 \\
p(x_3 = A \mid B, A) &= 0.5, & p(x_3 = B \mid B, A) &= 0.5 \\
p(x_3 = A \mid B, B) &= 0.05, & p(x_3 = B \mid B, B) &= 0.95
\end{aligned}$$

1a. *Greedy decoding* selects $x_t = \arg\max_x p(x \mid x_{<t})$ at each step. Compute the sequence $\mathbf{x}^{\text{greedy}}$ produced by greedy decoding, and report its joint probability.

1b. *Maximum a posteriori (MAP) decoding* returns $\mathbf{x}^\star = \arg\max_{\mathbf{x}} p(\mathbf{x})$. Enumerate all 8 sequences and report $\mathbf{x}^\star$ and its probability.

1c. The Viterbi algorithm finds the most-probable hidden-state sequence for a Hidden Markov Model in $\mathcal{O}(TK^2)$ time, where $K$ is the number of hidden states. Exact MAP decoding for a general autoregressive model of length $T$ with vocabulary size $V$ requires $\mathcal{O}(V^T)$ time.

Now consider a restricted autoregressive model in which each token depends only on the previous $k$ tokens: $p(x_t \mid x_{<t}) = p(x_t \mid x_{t-k}, \ldots, x_{t-1})$ for all $t > k$. For this restricted model, exact MAP decoding can be done in polynomial time using dynamic programming. You will design the algorithm.

Let $\delta_t(\mathbf{w})$ denote the probability of the most-probable sequence of length $t$ that ends in the window $\mathbf{w} = (x_{t-k+1}, \ldots, x_t)$.

(i) How many possible values of $\mathbf{w}$ are there?

(ii) Write the recursion that expresses $\delta_{t+1}$ in terms of $\delta_t$.

(iii) Using your recursion, state the total time to compute all $\delta_T$ values, as a function of $T$, $V$, and $k$. Verify that $k = 1$ recovers the $\mathcal{O}(TV^2)$ Viterbi bound, and that $k = T - 1$ recovers brute-force enumeration.

**Part 2: KV caching.**

A Transformer language model processes a sequence by computing, for each position $t$, a representation $h_t$ of the token at that position. From each $h_t$ it derives a key $K_t = W_K h_t$ and a value $V_t = W_V h_t$, using learned projection matrices $W_K$ and $W_V$. Causal self-attention then combines the keys and values from all positions $1, \ldots, t$ to produce the model's output at position $t$.

In autoregressive generation, the model starts from a prefix $x_1, \ldots, x_T$ and generates new tokens one at a time. To generate $x_{T+1}$, the model runs a forward pass and samples from the output distribution at position $T$. Then $x_{T+1}$ is appended to the sequence, and the model runs again to generate $x_{T+2}$. And so on, until $k$ new tokens have been generated.

In a naive implementation, each of the $k$ generation steps recomputes every $K_t$ and $V_t$ from scratch. But notice: the value of $K_t$ for a position $t$ already in the sequence does not change when a later token is appended. So we can compute each $K_t$ and $V_t$ just once and cache them, looking up the cached values on subsequent steps rather than recomputing. This optimization is called *KV caching* and is standard in every production language model inference system. You will analyze its computational benefit.

2a. Consider the naive generation procedure, in which each of the $k$ generation steps runs a full Transformer forward pass over the entire current sequence (no caching).

Let $N_{\text{naive}}(T, k)$ denote the total number of key computations $W_K h_t$ performed across all $k$ generation steps.

(i) Just before generating $x_{T+j}$, how long is the sequence the model is processing, and how many key computations does that forward pass require?

(ii) Sum your answer over $j = 1, \ldots, k$ to get a closed-form expression for $N_{\text{naive}}(T, k)$. Show that $N_{\text{naive}}(T, k) = \mathcal{O}((T + k)^2)$.

2b. KV caching relies on a specific property of causal self-attention. What is the property, and why does it hold? Why does the same caching break down under bidirectional self-attention?

2c. Now consider generation with KV caching. Before the first generation step, the model performs a single forward pass over the initial prefix $x_1, \ldots, x_T$ to compute and store $K_t$ and $V_t$ for each position $t = 1, \ldots, T$. This is called the *prefill* phase. Thereafter, at each of the $k$ generation steps, the model computes only the key and value for the single newly appended token, and reuses the cached keys and values for all earlier positions.

Let $N_{\text{cached}}(T, k)$ denote the total number of key computations $W_K h_t$ performed across the prefill and all $k$ generation steps.

(i) Give a closed-form expression for $N_{\text{cached}}(T, k)$.

(ii) State the ratio $N_{\text{naive}}(T, k)/N_{\text{cached}}(T, k)$ in the regime $k \gg T$ (long generations from short prompts).

2d. A single training forward pass over a sequence of length $k$ takes roughly the same wall-clock time as generating just one new token during inference - even though the training pass does far more total arithmetic.

Why does training parallelize across positions while generation does not? Does KV caching help during training?

**Part 3: Speculative decoding.**
Part 2 showed that generation is slow because it cannot be parallelized: each token must be sampled before the next can be computed. *Speculative decoding* is a technique that partially sidesteps this constraint by using two models: a large *target* model $p$ whose distribution we want to sample from, and a small *draft* model $q$ that approximates $p$ but is much cheaper to run.

Each round of speculative decoding proceeds as follows:

(i) **Drafting.** The draft model $q$ generates $k$ candidate tokens $x_{T+1}, \ldots, x_{T+k}$ autoregressively. This is cheap because $q$ is small.

(ii) **Verification.** The target model $p$ runs a single forward pass over the sequence $(x_1, \ldots, x_{T+k})$, producing the target distributions $p(\cdot \mid x_1, \ldots, x_{T+i-1})$ for every $i$ from 1 to $k+1$ in parallel. The first $k$ of these distributions are used to verify the drafts; the last one is a "bonus" distribution at position $T+k+1$, used in step (iii) if all drafts are accepted.

(iii) **Acceptance.** The draft tokens are considered one at a time. Draft token $x_{T+i}$ is accepted with probability

$$\min\left(1,\ \frac{p(x_{T+i} \mid x_1, \ldots, x_{T+i-1})}{q(x_{T+i} \mid x_1, \ldots, x_{T+i-1})}\right).$$

If $x_{T+i}$ is rejected, all subsequent drafts $x_{T+i+1}, \ldots, x_{T+k}$ are discarded, and a replacement token is sampled at position $T+i$ from a distribution constructed so that the output sequence still matches $p$.

Under this acceptance rule, the sequence of tokens output by speculative decoding has the same distribution as if we had generated from the target $p$ directly. Speculative decoding preserves the target distribution exactly.

3a. In the verification step, the target model produces $k$ conditional distributions in a single forward pass, while generating those same $k$ tokens from the target model alone would require $k$ sequential forward passes.

(i) What property of the target model's forward pass makes parallel verification possible? Why does the same property not enable parallel generation?

(ii) Suppose the draft model and the target model are identical, so $q = p$. What happens to speculative decoding? Is there any speedup, or any additional cost? Explain.

3b. The acceptance probability for draft token $x_{T+i}$ is

$$\min\left(1,\ \frac{p(x_{T+i} \mid x_1, \ldots, x_{T+i-1})}{q(x_{T+i} \mid x_1, \ldots, x_{T+i-1})}\right).$$

(i) When $p(x_{T+i} \mid x_1, \ldots, x_{T+i-1}) \geq q(x_{T+i} \mid x_1, \ldots, x_{T+i-1})$, what is the acceptance probability? Why does it make sense to always accept in this case?

(ii) When $p(x_{T+i} \mid x_1, \ldots, x_{T+i-1}) \ll q(x_{T+i} \mid x_1, \ldots, x_{T+i-1})$, what is the acceptance probability? Why does it make sense to usually reject?

## Solution

Your solution here.

**Problem 5** (Embedded Ethics, 10 pts)

One aim of the class session on Fairness in Model Selection was to show that we often need to consider the broader socio-technical context when using our CS expertise to solve real-world problems. This is just as true in the case of reinforcement learning. To practice this skill of considering the broader socio-technical context, answer the following prompt in a few sentences (at most 250 words). Answers should exhibit some careful thought about the socio-technical context, but don't need to be comprehensive.

Social media platforms like Facebook, TikTok, and X make extensive use of reinforcement learning, and many scholars have argued that widespread engagement with these platforms has contributed to political polarization, where users with a particular political leaning develop more extreme views over time. Assume users of a particular platform do in fact tend to develop more extreme political views over time. Using what you've learned about RL, devise a possible explanation for how the particular reward function the app's creators chose may have contributed to this problematic increase in political polarization.

## Solution

Your solution here.

## Name

## Collaborators and Resources

Whom did you work with, and did you use any resources beyond cs181-textbook and your notes?